AIA12: AI at Scale (MLOps)

Module Introduction and Theoretical Foundations

MLOps operationalizes **ML reproducibility, scalability, monitoring, and governance** for production systems handling millions of predictions daily. This module covers **CI/CD for ML**, **model monitoring**, **drift detection**, and **enterprise pipelines**—essential for Fortune 500 AI deployment.

MLOps Maturity Model:

text

Level 0: Manual scripts

Level 1: Automation

Level 2: Continuous training/deployment

Level 3: Experiment tracking

Level 4: Full self-service

Core Principles:

1. **Reproducibility**: Same code + data → same model
2. **Observability**: Model + data + system monitoring
3. **Reliability**: 99.9% uptime SLAs
4. **Governance**: Lineage, compliance, rollback

(Figure 1: MLOps maturity curve)

Section 1: ML Pipelines and Versioning

Pipeline Architecture:

text

Data → Feature Store → Training → Validation → Registry → Serving

MLflow Tracking:

python

```python
import mlflow
import mlflow.sklearn
from mlflow.tracking import MlflowClient


class MLExperiment:
    def __init__(self, experiment_name):
        mlflow.set_experiment(experiment_name)
```

```
        self.client = MlflowClient()

    def log_run(self, model, params, metrics, artifacts):
        with mlflow.start_run():
            # Log parameters
            mlflow.log_params(params)

            # Log metrics
            mlflow.log_metric("train_auc", metrics["train_auc"])
            mlflow.log_metric("val_auc", metrics["val_auc"])

            # Log model
            mlflow.sklearn.log_model(model, "model")

            # Log artifacts
            for name, path in artifacts.items():
                mlflow.log_artifact(path, name)

            # Log confusion matrix
            mlflow.log_figure(fig, "confusion_matrix.png")

        return mlflow.active_run().info.run_id

    def compare_runs(self, metric="val_auc"):
        runs = self.client.search_runs()
        best_run = max(runs, key=lambda x: x.data.metrics[metric])
        return best_run
```

DVC (Data Version Control):

text

```
dvc init
dvc add data/raw.csv
```

```
dvc push  # Remote storage
dvc repro  # Reproducible pipeline
```

Section 2: CI/CD for Machine Learning

ML-Specific GitOps:

text

```
Code → Tests → Data validation → Training → Validation → Staging → Prod
```

GitHub Actions + MLflow:

text

```
# .github/workflows/ml-cd.yml
name: ML CI/CD
on:
  push:
    branches: [main]

jobs:
  train:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.10'

    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install mlflow
```

```
  - name: Data validation
    run: python scripts/validate_data.py

  - name: Train model
    env:
      MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
    run: python train.py

  - name: Deploy to staging
    if: success()
    run: |
      mlflow models serve -m runs:/${{ steps.train.outputs.run_id }}/model -p 1234 &
      python scripts/test_staging.py
```

Section 3: Model Serving and Deployment Patterns

Serving Frameworks:

text

```
1. TensorFlow Serving: gRPC + REST
2. TorchServe: PyTorch native
3. vLLM/TGI: LLM optimized
4. KServe: Kubernetes native
```

Production Serving with KServe:

text

```
# kserve-model.yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: fraud-detection
spec:
  predictor:
    model:
```

```
    modelFormat:
      name: sklearn
    storageUri: gs://ml-pipeline/models/fraud-v2
    resources:
      limits:
        cpu: "2"
        memory: 4Gi
```

A/B Testing and Canary Deployment:

python

```python
class ModelRouter:
    def __init__(self, models, weights=[0.9, 0.1]):  # 90/10 traffic split
        self.models = models
        self.weights = weights

    def predict(self, request):
        model_id = np.random.choice(len(self.models), p=self.weights)
        return self.models[model_id].predict(request)
```

Section 4: Model Monitoring and Drift Detection

Drift Types:

text

```text
1. Concept Drift: P(y|x) changes
2. Data Drift: P(x) changes
3. Label Drift: P(y) changes
4. Model Drift: Performance degradation
```

Drift Detection Implementation:

python

```python
from alibi_detect.cd import TabularDrift
import evidently

class DriftMonitor:
```

```
def __init__(self, reference_data):
    self.reference_data = reference_data

    # Statistical tests
    self.drift_detector = TabularDrift(reference_data,
                                       p_val=0.05,
                                       preprocess_fn=normalize)

    # Evidently reports
    self.report = Report(metrics=[
        DataDriftPreset(),
        ModelPerformancePreset()
    ])

def detect_drift(self, current_data):
    # Statistical drift
    preds = self.drift_detector.predict(current_data)
    is_drift = preds['data']['is_drift']

    # Performance drift
    self.report.run(reference_data=self.reference_data,
                    current_data=current_data,
                    y_true=self.reference_labels)

    return {
        'statistical_drift': is_drift,
        'performance_metrics': self.report.as_dict()
    }

def alert(self, drift_detected):
    if drift_detected:
```

```
        trigger_retrain()
```

Monitoring Dashboard:

python

```python
import streamlit as st
import plotly.express as px

def monitoring_dashboard(drift_monitor, recent_data):
    st.title("🧠 Model Health Dashboard")

    col1, col2 = st.columns(2)
    with col1:
        drift_results = drift_monitor.detect_drift(recent_data)
        st.metric("Data Drift", "✅ No Drift" if not drift_results['statistical_drift'] else "🚨 DRIFT")

    with col2:
        perf = drift_results['performance_metrics']
        st.metric("AUC", f"{perf['auc']:.3f}")

    # Drift visualization
    fig = px.line(drift_history, x='timestamp', y='drift_score')
    st.plotly_chart(fig)
```

Section 5: Feature Stores and Data Management

Feature Store Architecture:

text

```text
Online: Low-latency point lookups
Offline: Batch training data
```

Feast Implementation:

python

```python
from feast import FeatureStore
from feast.repo_config import load_repo_config
```

```python
store = FeatureStore(repo_path="./feature_repo")

# Offline features for training
training_df = store.get_historical_features(
    entity_df=entity_df,
    feature_refs=[
        "driver_hourly_stats:conv_rate",
        "driver_hourly_stats:acc_rate"
    ]
).to_df()

# Online serving
features = store.get_online_features(
    features=[
        "driver_hourly_stats:conv_rate"
    ],
    entity_rows=[{"driver_id": 1001}]
)
```

Section 6: Enterprise Governance and Compliance

Model Registry with Provenance:

python

```python
class MLGovernance:
    def __init__(self):
        self.registry = {}

    def register_model(self, model_id, version, lineage):
        self.registry[model_id] = {
            'version': version,
            'lineage': lineage,  # data_versions, code_hash
```

```
        'metrics': training_metrics,
        'approval_status': 'pending'
    }

  def deploy_approved(self, model_id):
    if self.registry[model_id]['approval_status'] == 'approved':
        deploy_to_production(model_id)
```

Bias and Fairness Monitoring:

text

Demographic parity, Equalized odds

Continuous monitoring post-deployment

Section 7: Case Studies and Production Architecture

Case Study: Uber Michelangelo

text

1000+ models in production

Centralized feature store

Automated retraining pipelines

Complete Production Pipeline:

text

1. Data ingestion → Great Expectations validation
2. Feature store → Tecton/Feast
3. Training → Kubeflow + MLflow
4. Validation → A/B testing
5. Serving → KServe + Istio
6. Monitoring → Prometheus + Grafana

Capstone: End-to-End MLOps Platform

text

CI/CD → Feature store → AutoML → Monitoring → Governance